MÓDULO 5 — TOOLS, AGENTES, SKILLS Y MEMORIA · CLASE 10 DE 16

Skills, contexto y memoria

C9 construyó un agente capaz de decidir, llamar Tools, observar resultados y repetir. Hoy resolvemos el problema siguiente: cómo especializar ese agente sin convertir el prompt en un monolito y cómo darle continuidad entre ejecuciones sin confundir memoria con historial infinito ni con estado de workflow. Diseñamos Skills cargables, un contexto deliberado y memoria persistente con políticas explícitas de escritura y recuperación.

Skills · Progressive Disclosure · Dynamic Loading · Context Engineering · Working Memory · Conversational Memory · Persistent Memory · Compaction · Skill Anatomy · Memory Lifecycle · Write / Retrieve Policy · Provenance + TTL

Especializar

Encapsular procedimientos, recursos e instrucciones de dominio como Skills reutilizables.

Dosificar contexto

Cargar solo lo necesario para la tarea actual, en el momento adecuado.

Recordar con criterio

Persistir información útil entre sesiones mediante reglas explícitas de write/retrieve.

NÚCLEO OBLIGATORIO

Skills + contexto + memoria

Distinguir conceptos, diseñar una Skill, cargarla progresivamente y construir contexto con memoria relevante.

PRÁCTICA DE CONSOLIDACIÓN

Continuidad entre sesiones

Persistir memoria con aislamiento por usuario/sesión, recuperar solo lo pertinente y compactar conversaciones largas.

AMPLIACIÓN / LUEGO

Estado y reanudación

C11 agrega estado explícito, checkpoints, interrupciones y Human-in-the-Loop. Eso no es “más memoria”: resuelve otro problema.

Alcance de esta clase: vamos a diseñar Skills reutilizables, seleccionar cuánto de cada Skill cargar, construir contexto bajo presupuesto y tratar Memory como un subsistema gobernado: qué escribir, cómo recuperar, cómo resolver contradicciones y cuándo expirar información. En C11 se agrega estado/checkpoints para pausar y reanudar una ejecución concreta.

GUÍA

Pregunta central y conexión curricular

PREGUNTA CENTRAL

¿Qué problema aparece después de C9?

El runtime ya sabe usar Tools, pero el modelo todavía necesita saber **qué procedimiento especializado aplicar, qué contexto merece entrar en esta ejecución y qué información útil debería sobrevivir a la siguiente sesión**.

OBJETIVOS

Al finalizar deberías poder

- Diferenciar prompt, Tool, Skill, memoria y MCP.
- Diseñar Skills como paquetes reutilizables de instrucciones, recursos y procedimientos.
- Aplicar progressive disclosure y carga dinámica.
- Construir un contexto deliberado con presupuesto explícito.
- Distinguir memoria de trabajo, conversacional y persistente.
- Definir políticas de escritura, recuperación, compactación y aislamiento.
- Diseñar la anatomía de una Skill con metadata, instrucciones, ejemplos, recursos y criterios de activación.
- Explicar el ciclo de vida completo de una memoria: candidate → write → store → retrieve → rank → inject → update/expire.
- Distinguir memoria persistente de estado/checkpoint y justificar cuándo usar storage estructurado o semántico.

SECUENCIA

De C9 a C11

C9: el agente decide y actúa. **C10:** se especializa y conserva continuidad. **C11:** convierte la ejecución en un workflow persistente, supervisable y reanudable.

Hilo conductor único

Un buen agente no recibe “todo lo que sabemos” en cada llamada.
Selecciona capacidades, carga instrucciones especializadas, recupera memoria relevante y construye el contexto justo para decidir bien.

PARTE 1

Después de C9: un agente capaz todavía puede ser frágil

C9 resolvió acción; C10 resuelve especialización y continuidad

En C9 construimos el loop **Decide** → **Act** → **Observe**. Ese runtime puede invocar Tools, pero hay tres problemas nuevos cuando el sistema crece.

PROBLEMA 1

Prompt monolítico

Cada dominio agrega reglas, ejemplos y procedimientos. Si todo vive en el system prompt, el contexto crece y las instrucciones compiten.

PROBLEMA 2

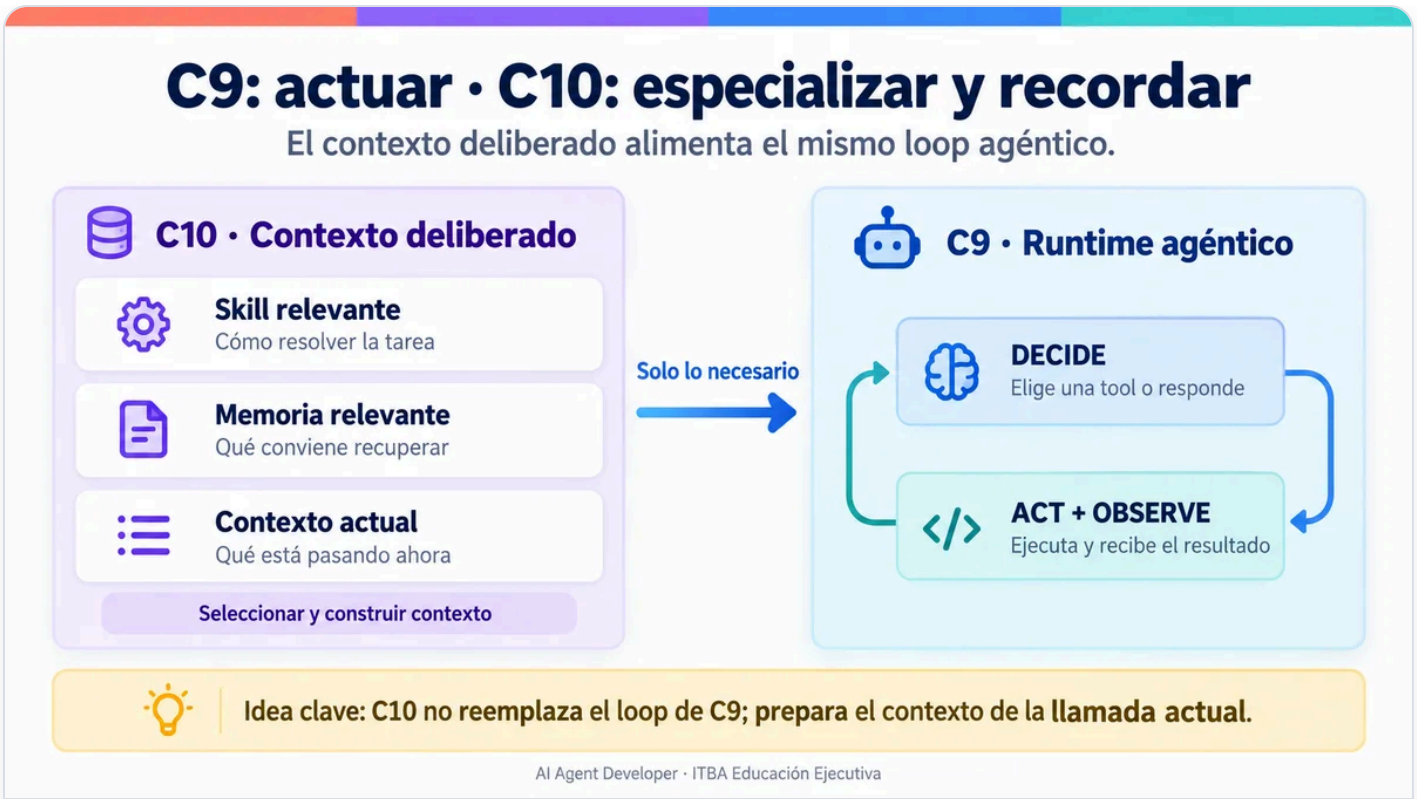
Amnesia entre sesiones

Una nueva ejecución puede ignorar decisiones o preferencias útiles de la anterior si nada se persiste deliberadamente.

PROBLEMA 3

Contexto indiscriminado

Guardar mucho no ayuda si luego recuperamos ruido, datos de otro usuario o información irrelevante para la tarea actual.



De C9 a C10: contexto deliberado con Skills y memoria para alimentar el loop agéntico.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

La continuidad entre el runtime de C9 y la preparación de contexto de C10.

CÓMO INTERPRETARLO

La inteligencia del agente depende tanto de la decisión del modelo como de qué contexto le entregamos antes de decidir.

DECISIÓN DE INGENIERÍA

No cargar todo por defecto. Seleccionar Skills y memoria según la tarea y el alcance de la ejecución.

Cambio de pregunta

En C9 preguntábamos “¿qué acción sigue?”. En C10 agregamos una pregunta previa:
“¿qué conocimiento operativo y qué memoria necesita el modelo para decidir bien esa acción?”

PARTE 2

Prompt, Tool, Skill, memoria y MCP: responsabilidades distintas

Cinco palabras que no conviene mezclar

En sistemas reales estos conceptos suelen aparecer juntos, pero resuelven problemas diferentes. Separarlos permite diseñar mejor y evita convertir “memoria” o “Skill” en nombres genéricos para cualquier texto que agregamos al prompt.

Concepto	Pregunta que responde	Ejemplo
Prompt	¿Qué instrucción recibe el modelo ahora?	“Respondé con evidencia y citá la fuente.”
Tool	¿Qué acción puede solicitar el modelo?	`search_course(query)`
Skill	¿Qué procedimiento/conocimiento especializado conviene cargar para resolver esta clase de tarea?	Skill “Revisión de contratos” con instrucciones, ejemplos, recursos y Tools asociadas.
Memoria	¿Qué información seleccionada del pasado debería influir en esta ejecución?	Preferencia de formato o decisión previa del usuario.
MCP	¿Cómo interoperan aplicaciones/agentes con capacidades externas mediante un protocolo común?	Un MCP Server que expone Tools/Resources. Se estudia en C13.

Regla mental de 10 segundos.

Prompt indica qué hacer ahora; una **Tool** permite ejecutar una capacidad; una **Skill** orienta cómo resolver una familia de tareas; **Memory** recupera información seleccionada del pasado; **State** —que aparece en C11— permite continuar una ejecución concreta; **MCP** estandariza cómo exponer o descubrir capacidades externas.



Prompt, Tool, Skill, memoria y MCP: piezas complementarias con responsabilidades distintas.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Cinco responsabilidades que suelen confundirse cuando un prototipo crece.

CÓMO INTERPRETARLO

Una Skill puede referenciar Tools; una memoria puede activar una Skill; MCP puede transportar capacidades. Pero siguen siendo conceptos distintos.

DECISIÓN DE INGENIERÍA

Nombrar cada pieza por el problema que resuelve y evitar “meter todo” en el system prompt.

¡ **“Skill” es una abstracción de diseño del curso.**
No asumimos que todos los proveedores implementen un estándar único llamado Skill. Para nosotros es un paquete reusable de instrucciones, recursos, ejemplos y procedimientos de dominio que puede cargarse según la tarea.

PARTE 3

Skills y Progressive Disclosure: especializar sin cargar todo

Skill no es “un prompt largo guardado en un archivo”

Una Skill encapsula una forma reusable de resolver una familia de tareas. En esta clase usamos **Agent Skills**, un formato abierto y portable donde cada Skill vive en una carpeta cuyo punto de entrada es `SKILL.md`.

DISCOVERY **name + description**	PROCEDIMIENTO **cuerpo de SKILL.md**
APOYO **references / scripts / assets**	EJECUCIÓN **Tools del runtime**

LECTURA GUIADA — TEXT

```
sales-analysis/
├── SKILL.md
├── references/
│   └── metric-definitions.md
├── scripts/
│   └── analyze-sales.py
└── assets/
    └── report-template.md
```

LECTURA GUIADA — MARKDOWN

```
---
name: sales-analysis
description: Analiza CSV de ventas, valida inconsistencias y calcula facturación.
metadata:
  course-version: "1.0.0"
  course-activation: "ventas|csv de ventas|facturación"
---

# Sales Analysis

1. Validar esquema y filas.
2. Leer definitions de métricas cuando hagan falta.
3. Calcular solo sobre filas válidas.
4. Generar gráfico e informe.
```

Qué es estándar y qué es nuestro

`name`, `description`, el cuerpo de `SKILL.md` y la organización en `references/`, `scripts/` y `assets/` siguen Agent Skills. Para la práctica agregamos metadata `course-*` que hace observable selección, versión y Tool hints. Esa metadata es una extensión del curso, no un requisito del estándar.

Demo concreta — análisis de ventas con Python.
Pedido: "Analizá este CSV de ventas. Mostrame la evolución mensual, los cinco productos con mayor facturación y si encontrás datos inconsistentes." La Skill `sales-analysis` contiene el procedimiento, definiciones de métricas, plantilla y un script auxiliar reutilizable. En modo local el Agent Loop usa ese script como código predefinido; en `LIVE_MODE=1` el modelo escribe su propio Python.

SKILL

Define el procedimiento

MODELO

Interpreta y escribe Python

TOOL

Ejecuta el código

RESULTADO

Métricas + gráfico + inconsistencias

Frontera importante

Agent Skills define un campo experimental `allowed-tools`, pero esta clase no lo usa para ampliar permisos. La autorización sigue perteneciendo al `ToolRegistry` y a `ToolPolicy` de C9. La Skill puede incluir o sugerir scripts y capacidades; no las ejecuta por sí misma.

La anatomía de una Skill profesional

Una Skill reusable necesita algo más que instrucciones. Con Agent Skills, `SKILL.md` combina metadata de discovery e instrucciones; el resto del material puede vivir en `references/`, `scripts/` y `assets/` para cargarse bajo demanda. El paquete puede versionarse sin convertir la Skill en una Tool.

Componente	Pregunta que responde	Qué evita
Frontmatter	¿Qué hace esta Skill y cuándo usarla?	Cargar el cuerpo completo solo para discovery.
Metadata opcional	¿Qué señales adicionales quiere usar nuestro loader?	Confundir extensiones del runtime con campos obligatorios del estándar.
Body de SKILL.md	¿Qué procedimiento seguir?	Reinventar la estrategia en cada turno.
Examples	¿Cómo se ve un resultado aceptable?	Pueden vivir en el body o en material de apoyo del paquete.
References / scripts / assets	¿Qué material adicional puede consultarse?	Inflar el contexto con plantillas o documentación innecesaria.
Tool hints	¿Qué capacidades suele requerir el procedimiento?	Confundir procedimiento con ejecución.
Version	¿Qué revisión de la Skill se usó?	Resultados imposibles de reproducir o auditar.

Carga progresiva de Skills: catálogo, instrucciones de la Skill elegida y recursos bajo demanda.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Las piezas internas de una Skill y su relación con el runtime del agente.

CÓMO INTERPRETARLO

Metadata sirve para descubrir; instrucciones y ejemplos para ejecutar; recursos y Tools se cargan o exponen solo si la tarea lo exige.

DECISIÓN DE INGENIERÍA

Versionar la Skill y mantener estable su responsabilidad aunque cambien recursos o Tools internas.

Una Skill también necesita frontera.

Si una Skill intenta describir toda la organización, todas las Tools y todas las excepciones, deja de ser una unidad reusable y vuelve a convertirse en un system prompt monolítico.

Progressive Disclosure

Si tenemos 40 Skills, no queremos inyectar el contenido completo de las 40 en cada llamada. El patrón de **progressive disclosure** revela información por etapas: primero metadata mínima para seleccionar; después las instrucciones de la Skill elegida; por último, recursos pesados solo si la tarea los necesita.

Context Builder: selección, priorización y organización del contexto bajo un presupuesto de tokens.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Tres niveles de información con costos de contexto distintos.

CÓMO INTERPRETARLO

La metadata sirve para elegir; las instrucciones sirven para ejecutar; los recursos sirven para profundizar.

DECISIÓN DE INGENIERÍA

Cargar una Skill por necesidad semántica, no porque exista en el repositorio.

Selección, competencia y fallback

En una aplicación con muchas Skills, seleccionar también es una decisión de arquitectura. Dos Skills pueden parecer relevantes para la misma consulta; ninguna puede ser suficientemente adecuada; o una tarea puede necesitar una Skill principal y un recurso secundario. La salida robusta no es “cargar todas por las dudas”.

Situación	Estrategia	Riesgo a evitar
Una Skill claramente dominante	Cargarla y reservar presupuesto para sus recursos.	Agregar Skills vecinas sin necesidad.
Dos Skills muy cercanas	Comparar activation criteria o pedir una señal adicional antes de cargar.	Instrucciones contradictorias dentro del mismo contexto.
Ninguna Skill aplica	Fallback al comportamiento base del agente.	Forzar una especialización incorrecta.

Tarea compuesta	Componer explícitamente o resolver por etapas.	Fusionar procedimientos sin orden ni prioridad.

LECTURA GUIADA

PYTHON

```python
candidates = skill_catalog.shortlist(query, limit=3)
selected = skill_selector.choose(query, candidates)

if selected.confidence < MIN_SKILL_CONFIDENCE:
    selected = None  # comportamiento base

if selected:
    context.add("skill", skill_loader.load(selected.name))
```

Qué mirar

La selección de Skill también debería ser observable: qué candidatas se consideraron, por qué se eligió una y qué versión fue cargada. Si una respuesta cambia después de actualizar una Skill, esa trazabilidad permite explicarlo.

Carga dinámica

LECTURA GUIADA

PYTHON

```python
catalog = skill_loader.list_summaries()
selected = select_skill(user_query, catalog)

skill = skill_loader.load(selected.name)
context.add_section("skill", skill.instructions)

if selected.needs_resource:
    context.add_section("resource", skill.load_resource(selected.resource))
```

Regla de diseño: si una Skill no es relevante para la tarea actual, su contenido no debería competir por atención dentro del contexto.

PARTE 4

Context Engineering: decidir qué entra, en qué orden y con qué presupuesto

No es “escribir un mejor prompt”

Context Engineering es diseñar el conjunto de información que estará disponible para el modelo en una llamada concreta: instrucciones, Skill seleccionada, mensaje actual, historia resumida, memoria recuperada, evidencia de Tools y recursos. Incluye selección, orden, formato, delimitación, compactación y descarte.

Puente con C7

En C7 el Context Builder elegía qué evidencia recuperada llegaba al LLM. En C10 generalizamos la misma idea: ahora no compiten solo chunks de RAG, sino también políticas, Skills, memoria, conversación, resultados de Tools y el pedido actual.
El principio no cambia: retrieved/available no significa automáticamente “entra al contexto”.

Fuente	Valor	Riesgo si se carga sin criterio
System / policies	Reglas estables	Conflictos si se mezclan con contenido no confiable
Skill	Procedimiento especializado	Instrucciones irrelevantes o contradictorias
Memoria	Continuidad	Datos viejos, incorrectos o de otro scope
Conversation	Coherencia del thread	Historia larga y ruido
Tool results / RAG	Evidencia	Contexto redundante o no confiable

Memoria de trabajo, conversacional y persistente, y su diferencia con el estado de un workflow.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Un pipeline de selección y compactación previo a cada llamada del modelo.

CÓMO INTERPRETARLO

Distintas fuentes compiten por un recurso finito: atención/context window.

DECISIÓN DE INGENIERÍA

Definir prioridades y presupuesto antes de llegar al límite de tokens.

El presupuesto se asigna; no se consume por accidente

Una ventana de contexto grande no implica que debamos llenarla. Conviene reservar capacidad para la salida y asignar presupuestos máximos por fuente. Los números dependen del modelo y de la tarea; el siguiente ejemplo es deliberadamente **ilustrativo**.

Políticas de memoria: criterios para guardar información y recuperarla cuando resulta relevante.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Una posible distribución de una ventana operativa entre políticas, Skill, Memory, historia, evidencia, pedido y salida.

CÓMO INTERPRETARLO

Las fuentes no tienen igual prioridad. La evidencia útil puede necesitar más espacio que una historia conversacional repetitiva.

DECISIÓN DE INGENIERÍA

Definir límites por fuente y una estrategia de degradación antes de llegar al hard limit del proveedor.

Más contexto puede producir una peor decisión

Supongamos que el usuario pregunta por una política vigente. El sistema recupera dos memorias antiguas, seis fragmentos RAG redundantes, una Skill completa con diez ejemplos y treinta mensajes del thread. Aunque todo “entre” en la ventana, la señal relevante queda diluida. **Context Engineering optimiza señal útil por token, no ocupación de ventana.**

Contexto acumulativo

“Metamos todo por seguridad”

- más ruido y duplicados;
- instrucciones viejas compiten;
- mayor costo y latencia;
- más difícil depurar qué influyó.

Contexto deliberado

“Solo lo que cambia la decisión”

- políticas estables preservadas;
- una Skill relevante;
- memoria filtrada y reciente;
- evidencia top-ranked;
- historia compactada.

Context rot: cuando más contexto empeora el sistema

Una ventana grande no elimina el problema de relevancia. Historial redundante, instrucciones viejas, resultados duplicados y memoria no pertinente pueden distraer al modelo. El objetivo no es llenar la ventana: es **preservar la información que cambia la decisión**.

Orden recomendado para reducir contexto

1. Eliminar duplicados y contenido claramente irrelevante.
2. Recuperar menos memoria/evidencia pero con mejor ranking.
3. Compactar historia conversacional manteniendo decisiones y hechos críticos.
4. Reducir ejemplos o recursos secundarios.
5. Preservar separadamente políticas e invariantes que no deberían perderse por resumen.

! **Compactar es una operación con pérdida.**
Un resumen puede omitir matices. No conviene usarlo como reemplazo de datos autoritativos ni de estado de workflow que debe reanudarse exactamente.

PARTE 5

Memoria: trabajo, conversación y persistencia

“Memoria” no es guardar todo el chat

La memoria útil es una política de selección. Conviene distinguir al menos tres formas de continuidad, porque cada una tiene distinta duración, costo y mecanismo de recuperación.

Tipo	Qué conserva	Duración típica	Ejemplo
Working memory	Información activa de la ejecución actual	Run / sesión	Resultado de una Tool que todavía será usado en el siguiente paso.
Conversational memory	Continuidad del thread mediante una ventana reciente y/o un resumen; no implica conservar todo el transcript	Conversación	Qué se acordó hace diez mensajes.
Persistent memory	Información seleccionada que debe sobrevivir nuevas sesiones	Entre sesiones	Preferencia estable del usuario o decisión de proyecto.

Arquitectura integrada: Skills, memoria y conversación preparan el contexto del loop agéntico de C9.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Tres memorias con distinta duración y, a la derecha, estado como concepto separado.

CÓMO INTERPRETARLO

Una conversación resumida puede ser memoria; no es automáticamente un checkpoint ejecutable.

DECISIÓN DE INGENIERÍA

Persistir recuerdos para continuidad y usar estado/checkpoints cuando necesitamos reanudar exactamente una ejecución.

¿Dónde persiste la memoria?

Persistir memoria no implica elegir siempre una base vectorial. El mecanismo depende de **qué tipo de dato guardamos y cómo esperamos recuperarlo**. Un store estructurado —por ejemplo SQLite o PostgreSQL— funciona bien para preferencias, hechos y metadata con schema; un índice vectorial puede ayudar cuando necesitamos recuperar recuerdos menos estructurados por similitud semántica.

Memoria estructurada

Campos y filtros explícitos

`user_id`, `kind`, `value`, `updated_at`, `source`. Ideal cuando sabemos qué dato buscamos y necesitamos aislamiento fuerte.

Memoria semántica

Recuperación por significado

Útil para notas o recuerdos menos estructurados. Requiere filtros de namespace, ranking y límites para evitar traer recuerdos irrelevantes.

Regla de diseño: elegir el store por la semántica del dato y la consulta esperada, no porque "memoria de IA" implique automáticamente una base vectorial.

Profundización opcional

Comparar ambos enfoques con el mismo conjunto pequeño de recuerdos. El objetivo no es declarar un ganador, sino observar qué consultas se resuelven mejor con filtros exactos y cuáles se benefician de similitud semántica.

La anatomía de un recuerdo persistente

Guardar solo `key=value` alcanza para una demo, pero no para gobernar memoria real. Un recuerdo útil necesita contexto sobre **qué significa, de quién es, de dónde salió, desde cuándo vale y cuándo debería revisarse**.

Campo	Función
`kind`	Distingue preferencia, hecho, decisión de proyecto, restricción, resumen o episodio.
`value`	Contenido normalizado del recuerdo.

`scope`	Usuario, sesión, curso, proyecto u otra frontera de aislamiento.
`source`	Declaración explícita, Tool, documento, inferencia u otra procedencia.
`created_at / updated_at`	Permite razonar sobre vigencia y cambios.
`confidence`	Señala si el dato es confirmado o tentativo; no reemplaza verificación.
`expires_at`	TTL opcional para información temporal.
`supersedes`	Vincula una nueva versión con el recuerdo que reemplaza.

LECTURA GUIADA

JSON

```json
{
  "kind": "preference",
  "value": "prefiere ejemplos de Python",
  "scope": {"user_id": "u123", "course_id": "ai-agent-dev"},
  "source": "explicit_user_statement",
  "updated_at": "2026-09-12T16:00:00-03:00",
  "confidence": 1.0,
  "expires_at": null,
  "supersedes": null
}
```

Provenance no es decoración.

Si dos recuerdos se contradicen, la fuente y la fecha ayudan a decidir qué hacer. Un dato explícitamente confirmado por el usuario no debería tratarse igual que una inferencia débil de una conversación antigua.

PARTE 6

Políticas de memoria: escribir poco, recuperar mejor y aislar correctamente

La memoria necesita dos gates

Un sistema con memoria tiene al menos dos decisiones independientes: **qué merece persistirse** y **qué merece recuperarse ahora**. Guardar todo y recuperar todo solo traslada el problema del historial al store persistente.

Anatomía de una Skill reutilizable: identidad, activación, procedimiento, recursos y dependencias.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Un gate al escribir y otro al recuperar.

CÓMO INTERPRETARLO

La memoria puede existir en el store y aun así no ser pertinente para la consulta actual.

DECISIÓN DE INGENIERÍA

Definir criterios de write y retrieve antes de acumular datos persistentes.

El ciclo de vida completo de Memory

Memory no es solo persistencia. Es un pipeline con decisiones antes y después del store. Un buen diseño puede descartar candidatos, normalizar un dato, recuperarlo solo dentro del scope correcto, rankearlo contra otros recuerdos y finalmente actualizarlo o dejarlo expirar.

Ejemplo de presupuesto de contexto de 16k tokens: asignación por fuente, prioridades y reserva de salida.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Ocho pasos desde un candidato de memoria hasta su actualización o expiración.

CÓMO INTERPRETARLO

Persistir es solo un paso. Recuperación, ranking, scope y vigencia determinan si ese recuerdo será útil o dañino.

DECISIÓN DE INGENIERÍA

Medir y depurar Write Gate y Retrieve Gate por separado: pueden fallar de maneras distintas.

Contradicciones, vigencia y actualización

Los recuerdos cambian. Un usuario puede modificar una preferencia; un proyecto puede migrar de base de datos; una política puede vencer. El sistema no debería acumular ambas versiones como si fueran igualmente válidas ni depender de “la última cadena que apareció”.

Caso	Respuesta razonable
Nueva declaración explícita contradice una preferencia previa	Marcar la anterior como superseded y conservar trazabilidad del cambio.
Dato temporal supera su fecha útil	Aplicar TTL / expires_at y excluirlo de recuperación normal.
Dos fuentes confiables discrepan	No fusionar silenciosamente; conservar provenance y elevar incertidumbre.
Inferencia débil contradice un dato confirmado	Priorizar el dato confirmado; la inferencia no debería reemplazarlo automáticamente.

> ! **Memory no es ground truth.**
> Un recuerdo persistido sigue siendo dato sujeto a vigencia, fuente y contexto. Para decisiones críticas puede requerir revalidación contra un sistema autoritativo.

Qué conviene recordar

Candidato	¿Persistir?	Por qué
“Prefiero respuestas con ejemplos de Python.”	Sí, si el usuario lo desea	Preferencia estable que puede mejorar futuras sesiones.
“Hoy estoy probando un mock de API.”	Probablemente no	Dato temporal de la sesión.
“El proyecto usa PostgreSQL 15.”	Puede corresponder	Decisión durable del proyecto, idealmente con fuente y fecha.
Resultado crudo de una búsqueda de 20 chunks	No como recuerdo estable	Es evidencia temporal; puede regenerarse y consume mucho.

> ! **Minimización: recordar menos también es una feature.**
> No conviene persistir secretos, credenciales, datos sensibles innecesarios, resultados crudos de Tools ni información temporal solo "por si acaso". Una política de memoria debe permitir actualizar, expirar o eliminar recuerdos cuando dejan de ser válidos o pertinentes.

Aislamiento por usuario y sesión

Un store persistente debe tener namespace explícito. Como mínimo, diferenciar `user_id` y, cuando corresponda, `session_id`, `course_id` o `project_id`. La similitud semántica nunca debe saltarse esa frontera.

LECTURA GUIADA PYTHON

```python
memory_store.write(
    namespace={"user_id": user_id, "project_id": project_id},
    kind="preference",
    value="prefiere ejemplos de Python",
    source="explicit_user_statement",
)

memories = memory_store.search(
    namespace={"user_id": user_id, "project_id": project_id},
    query=current_query,
    limit=4,
)
```

> ! **El filtro de namespace es parte de la política, no una sugerencia al modelo.**
> No deberíamos recuperar memorias de otros usuarios y esperar que el LLM "se dé cuenta" de que no corresponden.

Compactación de conversaciones

Cuando un thread crece, podemos mantener una ventana reciente y un resumen acumulativo. La compactación debería preservar objetivos, decisiones, restricciones, pendientes y referencias necesarias; no simplemente "hacer el texto más corto".

LECTURA GUIADA PYTHON

```python
context = [
    system_policy,
    loaded_skill,
    retrieved_memories,
    conversation_summary,
    *recent_messages,
```

```
    current_user_message,
]
```

Caso end-to-end — la misma persona, dos sesiones

El valor de Memory se entiende mejor cuando la segunda ejecución puede beneficiarse de la primera sin reconstruir todo el chat. El ejemplo siguiente combina Skill, Tools y Memory manteniendo separadas las responsabilidades.

Ciclo de vida de la memoria persistente: selección, almacenamiento, recuperación, uso y mantenimiento.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

Una preferencia explícita se convierte en candidato de memoria y reaparece selectivamente en una sesión futura.

CÓMO INTERPRETARLO

La continuidad no requiere conservar cada token del pasado: requiere persistir y recuperar las señales que siguen siendo útiles.

DECISIÓN DE INGENIERÍA

Guardar la preferencia separada de la conversación y recuperarla solo si el scope y la tarea la vuelven pertinente.

Preguntas para leer el caso

1. ¿Qué parte pertenece a la Skill y cuál a Memory?
2. ¿Qué dato de la sesión 1 no deberíamos persistir?
3. ¿Qué ocurre si en la sesión 2 el usuario dice “ya no quiero ejemplos de Python”?
4. ¿Cómo cambia el diseño si esa preferencia aplica solo a este curso y no globalmente?

Arquitectura integrada

Continuidad entre dos sesiones mediante una Skill y la recuperación de una preferencia persistida.. La trazabilidad comienza en la ingesta y continúa hasta las métricas.

CÓMO LEER ESTE VISUAL

QUÉ MUESTRA

La arquitectura completa de C10 envolviendo, pero no reemplazando, el runtime de C9.

CÓMO INTERPRETARLO

Skill Store y Memory Store son fuentes distintas del Context Builder; el loop sigue usando Tools como antes.

DECISIÓN DE INGENIERÍA

Separar preparation, execution y persistence para poder probar y gobernar cada etapa de forma independiente.

NOTEBOOK

Notebook guiada — Skill loader, context budget y memoria persistente

PRÁCTICA TÉCNICA

De una Skill concreta a la arquitectura completa

La notebook comienza con una Agent Skill real en formato `SKILL.md` para analizar un CSV de ventas —incluyendo `references/`, `scripts/` y `assets/`— y luego recorre selección, Progressive Disclosure, presupuesto de contexto y el ciclo de memoria.



Context Builder

budget + compaction

↓

04 · C9

Agent Loop

Tools + Policy

Recorrido sugerido

SKILL

Ver primero una capacidad concreta

Analizar ventas con instrucciones, recursos y una Tool de Python; después generalizar selección, carga y fallback.

CONTEXTO

Presupuesto explícito

Medir y limitar cuánto ocupan Skill, memoria, resumen, mensajes recientes y evidencia.

MEMORIA

Persistencia entre sesiones

Guardar un recuerdo seleccionado, cerrar la sesión y recuperarlo desde otra ejecución con el mismo namespace.

DEFINITION OF DONE

- [ ] Puedo descubrir Agent Skills leyendo frontmatter sin cargar el body ni sus recursos.
- [ ] La Skill correcta se carga dinámicamente según la consulta.
- [ ] Puedo inspeccionar cuánto contexto ocupa cada sección.
- [ ] La memoria persistida sobrevive a una nueva ejecución.
- [ ] Una memoria de otro user_id/project_id no aparece en la recuperación.
- [ ] Una conversación larga se compacta sin perder objetivos y decisiones críticas.
- [ ] Puedo explicar por qué una Skill fue seleccionada y qué versión se cargó.
- [ ] Un recuerdo persistido conserva scope, source y vigencia suficiente para poder gobernarlo.

- [ ] Puedo simular dos sesiones donde la segunda recupera memoria relevante sin pegar todo el historial anterior.

ACTIVIDAD

Skill & Memory Architecture Review — especializar y recordar sin contaminar contexto

TRABAJO GRUPAL

Diseñá una Skill y su política de memoria para un agente que ya sabe usar Tools

Cada grupo recibe un caso de negocio. Debe decidir qué conocimiento pertenece a la Skill, qué merece persistir como Memory, qué debe quedar fuera y cómo construir el contexto de una segunda sesión. Otro grupo revisará ambigüedades, sobre-memorización y scope incorrecto.

1 · SKILL

Paquete reusable

- SKILL.md + name
- description + metadata
- body de instrucciones
- references/scripts/assets

2 · MEMORY WRITE

Qué persiste

- candidate memories
- source/provenance
- scope
- TTL / supersedes

3 · MEMORY RETRIEVE

Qué vuelve

- scope filter
- relevance/recency
- top-k / budget

- contradictions

4 · CONTEXT

Segunda sesión

- qué Skill cargar
- qué Memory recuperar
- qué descartar
- orden final del contexto

Canvas de entrega

Elemento	Decisión del grupo	Cómo probaría que funciona
SKILL.md / name / course-version		
Discovery description / activation metadata		
SKILL.md body / instrucciones críticas		
references / scripts / assets on-demand		
Memory candidates		
Write policy + provenance		
Scope + TTL		
Retrieve / ranking policy		
Contradicción / supersede		
Context budget sesión 2		

Red-team obligatorio

El equipo revisor debe probar al menos cuatro fallas: Skill equivocada, memoria contradictoria, recuerdo de otro namespace y contexto que excede el presupuesto. El diseño debe explicar cómo detecta o limita cada caso.

RECURSOS

Para profundizar sin adelantar C11

ARQUITECTURA

Skills

Paquetes reutilizables de instrucciones, procedimientos, ejemplos y recursos que pueden cargarse dinámicamente.

Specialization · Reuse

CONTEXTO

Context Engineering

Selección, priorización, delimitación y compactación del contexto disponible para cada decisión del modelo.

Budget · Compaction

MEMORIA

Persistent Memory

Store con políticas explícitas de escritura, recuperación y aislamiento por usuario/sesión/proyecto.

SQLite · Namespaces

PRÓXIMA CLASE

Estado + HITL

C11 agrega checkpoints, pausa/reanudación y aprobación humana. La memoria de C10 no reemplaza esos mecanismos.

C11 · LangGraph

CIERRE

Qué tiene que quedar de la Clase 10

01

Una Skill es procedimiento reusable

Metadata, activación, instrucciones, ejemplos, recursos y versión forman una unidad especializada.

02

Progressive Disclosure reduce ruido

Descubrir con metadata, cargar una Skill y abrir recursos solo cuando hacen falta.

03

El contexto se diseña

Policies, Skill, Memory, historia y evidencia compiten por un presupuesto finito.

04

Memory tiene ciclo de vida

Candidate, write, store, retrieve, rank, inject y update/expire son decisiones distintas.

05

Provenance y scope importan

Un recuerdo sin fuente, vigencia o aislamiento puede dañar más que ayudar.

06

Memory no es State

Memory mejora futuras decisiones; C11 usará estado/checkpoints para continuar una ejecución concreta.

Próxima clase

Clase 11 — Estado y Human-in-the-Loop: cómo persistir una ejecución, interrumpirla, pedir aprobación y reanudarla de forma controlada.
